



UNIVERSIDAD
COMPLUTENSE
MADRID

Sistema Inteligente de Intercambio de Recursos con IA (OLLAMA)

Integrantes:

**ALONSO CAMPILLO MARTÍNEZ
ERICKA DEL VALLE BRACHO PÉREZ
JIAHUI YOU
LUIS ÁNGEL GARCÍA ROJAS
SHOMARA DEYANIRA ACOSTA SANTANA
XIN XIANG LIN ZHOU**

Madrid, Marzo 2026

1. Introducción

El sistema desarrollado consiste en un agente automático capaz de negociar recursos con otros jugadores dentro de un entorno de intercambio similar al juego Catán. Su objetivo es simular el comportamiento de un jugador real, manteniendo conversaciones y tomando decisiones de intercambio de forma autónoma.

Para ello, el sistema utiliza un modelo de lenguaje ejecutado mediante Ollama, que permite interpretar mensajes, generar respuestas coherentes y detectar cuándo se ha alcanzado un acuerdo entre jugadores.

Además, el agente no solo se limita a la conversación, sino que también es capaz de ejecutar intercambios reales en el servidor del juego, integrando la comunicación con la lógica del sistema.

2. Funcionalidades principales

El sistema implementa las siguientes funcionalidades:

- Recepción de mensajes desde otros jugadores.
- Inicio automático de conversaciones cuando es necesario.
- Gestión independiente del historial de cada jugador.
- Consulta de recursos disponibles y objetivos del juego.
- Generación automática de respuestas mediante inteligencia artificial.
- Detección de acuerdos durante la negociación.
- Ejecución y registro de intercambios en el servidor.

3. Funcionamiento general

El agente actúa como un jugador autónomo dentro del sistema. Cuando recibe un mensaje, lo analiza teniendo en cuenta el contexto de la conversación y la información actual del juego, y genera una respuesta adecuada.

En caso de que se alcance un acuerdo, el sistema identifica la situación y ejecuta el intercambio correspondiente, enviando el intercambio al servidor.

El funcionamiento se basa en la integración de tres componentes principales:

- El servidor del juego, que proporciona información sobre recursos, objetivos y jugadores activos.
- La aplicación desarrollada con FastAPI, que gestiona la comunicación entre jugadores.

- El modelo de lenguaje ejecutado en Ollama, responsable de la toma de decisiones y generación de respuestas.

4. Tecnologías utilizadas

El desarrollo del sistema se ha realizado con las siguientes tecnologías:

- Python como lenguaje principal.
- uv como gestor de entorno y dependencias.
- FastAPI para la creación de la API de comunicación.
- Requests para la realización de peticiones HTTP.
- Threading para la gestión de múltiples conversaciones simultáneas.
- Ollama como entorno de ejecución del modelo de lenguaje.
- Modelo de lenguaje minstral-3:8B, utilizado para generar respuestas y detectar acuerdos de intercambio.
- JSON como formato de intercambio de datos entre componentes.

5. Estructura del sistema

El sistema se organiza en varios componentes principales:

- Agente (FastAPI), encargado de gestionar la comunicación y la lógica del sistema.
- Modelo de lenguaje (Ollama), responsable de generar respuestas y detectar acuerdos.
- Servidor del juego, que proporciona información y registra los intercambios.
- Otros jugadores, con los que se establece la comunicación.

Estos componentes interactúan entre sí para permitir el funcionamiento completo del sistema.

6. Estructura del proyecto

El proyecto se organiza en varios directorios y archivos principales:

- app: contiene la configuración principal de la aplicación.
- routes: define los endpoints de la API que permiten la comunicación entre jugadores.
- services: incluye la lógica del sistema, como la gestión de conversaciones y la interacción con el modelo de lenguaje.
- run.py: archivo principal que lanza la aplicación.
- pyproject.toml y uv.lock: gestionan las dependencias y el entorno del proyecto.

Esta organización permite separar la lógica del sistema, la comunicación y la configuración, facilitando el mantenimiento y la escalabilidad del proyecto.

7. Funcionamiento del sistema

7.1 Recepción de mensajes

El sistema expone un endpoint /buzon que permite recibir mensajes de otros jugadores. Cada mensaje incluye la IP del remitente, lo que permite identificar la conversación correspondiente. La información se almacena en el historial y se actualiza el estado de actividad del jugador.

7.2 Comunicación automática

El sistema incluye un proceso en segundo plano que permite su funcionamiento autónomo. Este proceso actualiza la lista de jugadores activos, detecta posibles situaciones de inactividad y puede iniciar nuevas conversaciones cuando no existe historial previo.

De esta forma, el agente no solo responde a mensajes, sino que también actúa de forma proactiva.

7.3 Gestión de conversaciones

Cada jugador dispone de un historial independiente, lo que permite mantener conversaciones coherentes y separadas. Además, el sistema controla el estado de cada interacción para determinar si la negociación sigue en curso o ya ha finalizado.

7.4 Consulta de información del juego

Antes de generar una respuesta, el sistema consulta el estado actual del juego, incluyendo los recursos disponibles y el objetivo. Esto garantiza que las decisiones del agente se basen en información real y actualizada.

7.5 Generación de respuestas y ejecución de acciones

Las respuestas se generan mediante el modelo de lenguaje, que analiza el contexto de la conversación y la situación del juego.

Además de generar texto, el modelo es capaz de devolver información estructurada en situaciones específicas, que el sistema interpreta para ejecutar acciones como el cierre de un intercambio. En ese caso, el sistema procesa dicha información y ejecuta automáticamente el intercambio correspondiente, enviándolo al servidor.

7.6 Envío de mensajes

Una vez generada la respuesta o ejecutado el intercambio, el sistema envía el mensaje al otro jugador. Dependiendo del estado de la negociación, puede tratarse de una respuesta intermedia o de una confirmación final.

6. Flujo de funcionamiento

El proceso completo sigue los siguientes pasos:

1. El sistema puede iniciar comunicación enviando mensajes a otros jugadores o esperar a recibir uno.
2. El sistema identifica al remitente mediante su IP.
3. El mensaje se almacena en el historial.
4. Se consulta el estado actual del juego.
5. El modelo genera una respuesta.
6. El sistema envía la respuesta al jugador.
7. Si se alcanza un acuerdo, se ejecuta y envía el intercambio al servidor.

En la siguiente figura se muestra el flujo básico del sistema, donde se representan los principales componentes y su interacción.

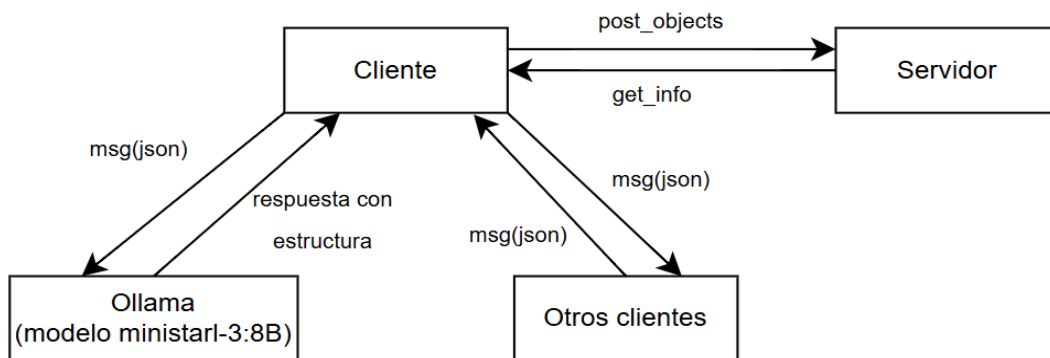


Figura 2. Flujo detallado del sistema

A continuación, se presenta una versión más detallada del flujo, donde se describe con mayor precisión el proceso de comunicación y ejecución de acciones dentro del sistema.

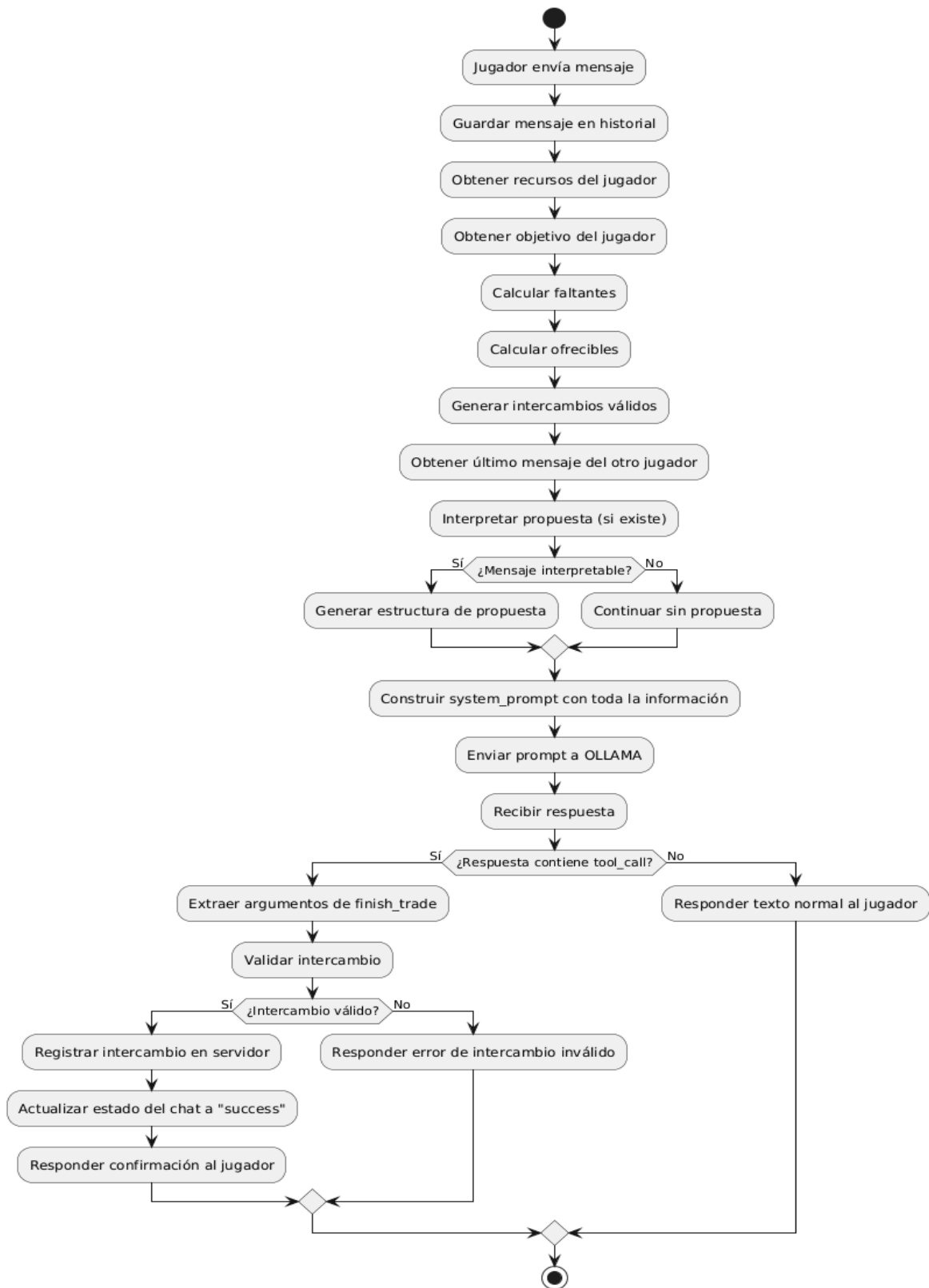


Figura 2. Flujo detallado del sistema

7. Requisitos y ejecución

Para la ejecución del sistema es necesario:

- Disponer de Python instalado.
- Tener configurado uv para la gestión de dependencias.
- Instalar las dependencias del proyecto.
- Contar con el servidor del juego en funcionamiento.
- Tener Ollama instalado y ejecutando el modelo seleccionado.
- Disponer de conectividad entre los jugadores.

Pasos básicos de ejecución:

- Iniciar el servidor del juego.
- Ejecutar Ollama con el modelo correspondiente.
`bin/ollama run ministral-3:8B`
- Lanzar la aplicación FastAPI.
`uv run run.py`
- Verificar el registro del agente.
- Iniciar la interacción entre jugadores.

8. Conclusión

El sistema desarrollado integra distintos componentes como la gestión del juego, la comunicación entre jugadores y el uso de un modelo de lenguaje para automatizar la negociación de recursos.

El agente es capaz de interpretar mensajes, generar respuestas coherentes y ejecutar intercambios cuando se alcanza un acuerdo, funcionando de manera autónoma dentro del entorno del juego.

En general, el sistema demuestra que es posible aplicar modelos de lenguaje en escenarios interactivos de forma práctica, permitiendo automatizar procesos de negociación de manera eficiente.